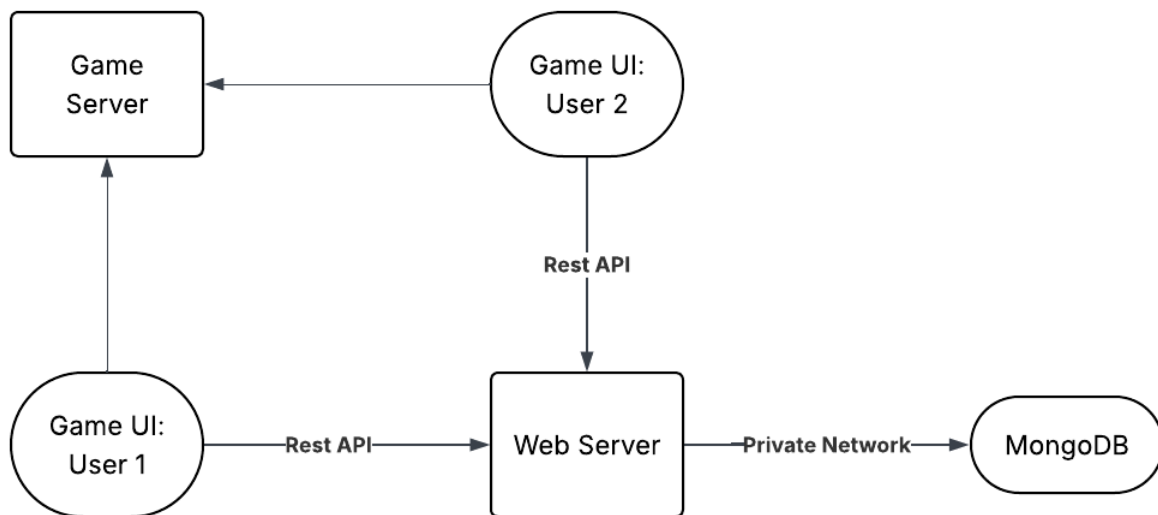


CSC581-Final- Project: Online Connect 4

By Seth Conley

Chapter 1: Overview

My Project is a multiplayer online Connect 4 game. The application allows 2 players to play a game of Connect 4 online, with the server being able to host more than 1 match at a time. Users can create accounts that store their wins, losses and draws, and those stats are updated on the home page after every game. The application runs on three containers: a nodejs container that serves the frontend files and connects to the database, a mongo image with the schema applied from the frontend server, and a second nodejs container that runs a websocket server which enables users to actually play the game. The project was constructed to be containerized from the beginning, so its structure was defined by a three container layout.



Database Container

The database service runs off of the base mongo image, which simplifies the process of using a database because I didn't need to start up a mongo server. I chose mongo for the project because of its ease of use with nodejs with the mongoose module, and the convenience of the official image. The database only needed one schema, that being the user account. The web server communicates with the database over the default private

docker compose network, using a url with the required port for the database. The game server does not connect to the database.

Frontend Container

The web-server service is built off of the base official nodejs image, with all of the necessary file added during the build process. I chose node because the use of express made routing easy. The server is organized in the Model View Controller framework. The web server handles every feature of the application other than connecting the users to each other, and hosting the actual database. It serves three main pages: A login screen, an account creation screen, and the home screen, which is where the game is played and shows the users stats.

Game Server

The game-server service is built off of the base official nodejs image, but unlike the web-server it only needs app.js and the package file. The game server supplies all the back end functionality of the actual game of connect 4. In order for the two users to connect, the server must link them together and pass data between the two. Web sockets are the best way for the application to create a persistent connection between two players.

Chapter 2: Implementation

Database

As discussed in chapter 1, the database container needs minimal editing: just the schema and connection to the backend container.

Frontend

When a new user creates an account, the registration page sends a request to the server with the new username and password, which is routed to the registration controller. The controller queries the database to see if there is already a user with the requested username, and return a 409 status if a conflict exists. If the username is not taken, the controller uses the User model to save the new account into the database container. When the user attempts to log in, the web server receives a request with the username and password, and checks them with a database query. If the password is correct, the user gets a cookie which is required to pass the authentication middleware that runs before the home page.

The home page displays three main elements. A div for the game on the left, a chatbox in the middle with a turn indicator above it, and a stats display on the right. When the user is not in a game, the div contains a "look for game" button, and the turn indicator states that the user is not in a game.

When the page is first loaded, the front end javascript sends a post request to the web server with the players username. This request is routed to "/stats", which is not covered by the authentication middleware because the stats controller runs its own authentication based on the users cookie before sending back their stats. On a successful query, the stats are then displayed on the right. Every time a game ends, the javascript queries the database with an update that increments the appropriate stat for a win, loss, or draw. A majority of the javascript is devoted to running the actual game by communicating with the game server.

Web Socket Container

The container uses the "ws" node module, which allows node to host a websocket server. A websocket consists of the server and a client. When a client is connected, the server creates a websocket instance, which is unique to each user. The server can send data to the user and the user can send data back, but only to their websocket instance. This creates a pairing of one user to one instance. There cannot be multiple users per instance. In order to connect the two users and their instances, I created a global map to store client instances that every instance can access. When a user clicks the start game button, the client side javascript connects to the websocket server and sends a "setup" json package containing their username. The server receives the message and adds the instance to the list in the key/value pair (username, websocket instance). It then checks a global list called waiting. If there is no one waiting for a second person to play, the server pushes the username to the list. If there is someone waiting, the server removes the first username from the list and initiates a sort of handshake.

The end goal of the handshake is for both users websocket instances to synchronize the game, including what color each player is, who is going first and subsequently whos turn it is, the game board and number of moves played. The instance that initiated the handshake decides every aspect of the initial game state. It creates a game object with an empty board, a stored array of rows to simplify the turn logic, and a number of moves. It then adds this game object to another global map of game objects with the key/value pair (gameID, game object). The gameID is a randomly generated number. It then creates two setup objects, one for the current instance' client, and the other to be sent to the other instance. The setup object for the client tells the javascript what the players own color is and if their turn is first, along with the other players color by process of elimination. The setup object for the other instance can only be sent to the other client, and not the other instance directly. When the setup object is received by the other client, the client side javascript parses its own color and turn, before sending the object to its own instance. This setup object contains the initiators username, allowing both instances access to the other via the map, along with the game ID created by the initiator so both instances can edit the same game object.

During the game, the instances communicate with the clients and each other via json packages containing a label field along with its necessary data, which is then parsed on the other side based on chaining if-else statements. For instance, if the json contains a

"message" field, the client side javascript will display that message to the chat. Any time a user makes a move, their client javascript sends the move to their instance, which updates the game object and sends the move to the other players client. Only the instance of the player that took the move updates the game board and checks to see if they've won in order to prevent a desync.

When a player does win, the server sends a json object telling the other client and its instance that they have lost. Both instances then reset their variables storing the other username, turn boolean, color variables, and gameId before removing the game from the game list. On the client side, the winner and the loser both query the database through the webserver to add the win/loss and update the stats page, before being disconnected from the websocket server and reverting the home page to its "not in game" state.

If a user disconnects from the server in the middle of a game, the server and client both detect this, and penalize the disconnected player with a loss, and credit the remaining player with a win before running the usual win/loss procedures.

Seth Conley

610-235-7318 | Sethconley82@gmail.com | www.linkedin.com/in/seth-conley-62293b344

Education – West Chester University

Bachelor of Science - Computer Science: Graduated May 2025 Summa Cum Laude

Masters of Science – Computer Science: Expected September 2025 – May 2027

Academic Achievements:

- Dean's list 7 semesters
- Membership in the computer science honors society Upsilon Pi Epsilon
- Placed third in the West Chester University Programming Competition

Relevant Coursework

- Computer Science 1-3
 - Learned coding basics such as variable types, loops and methods/functions.
 - Developed an understanding of how to translate a solution from spoken language into code.
 - Created programs like a recursive depth-first maze solving algorithm that utilized the concepts we learned in class while applying clean code practices.
- Data Structures and Algorithms
 - Learned data structures like Stack, Queue, and Hash maps.
 - Learned what makes a program efficient through runtime analysis using Big O notation.
 - Learned what makes algorithms effective, efficient, and complete.
 - Wrote code implementing graph exploration algorithms.
- Software Engineering
 - Learned advanced Object-Oriented Programming principles.
 - Learned how to write readable code.
 - Eliminated redundant code with methods where appropriate.
 - Utilized the SOLID design principles.
 - Applied various design patterns to make code that is extendable yet not modifiable.
- Computer Systems
 - Learned about computing operations at the bit level.
 - Wrote C code that demonstrated how data is stored in memory and how it can be manipulated.
 - Studied assembly code.
 - Learned how CPUs function.
 - Discussed the Memory Hierarchy of RAM and CPUs.
- Big Data Engineering
 - Learned the MapReduce programming paradigm
 - Worked in spark using a Docker image provided by our professor to simulate multiple machines working in parallel.
 - Learned multiple methods of parallelizing statistical analysis.
 - Practiced developing programs by replicating algorithms using Python.
 - Wrote a technical paper analyzing a large dataset.
- Topics in Computer Security
 - Learned about cybersecurity on machines running the ARM instruction set.
 - Analyzed assembly code in order to write programs that took advantage of vulnerabilities specific to the ARM instruction set.
 - Utilized Stack Overflow attacks, Return Hijack Attacks, and Shellcode Attacks.
 - Learned how to use "gadgets" to execute a Return Oriented Programming Attack.
 - Worked with a group to complete a "Capture the Flag" style activity for the final project.

Projects

- Paper Presentation: *AI/ML for Network Security: The Emperor has no Clothes*
 - Presented a technical paper with a group to the class.
 - Portrayed the key points of the paper without unnecessary technical detail.

- Received an A and feedback from the professor complementing the presentations clarity.
- Analysis of Steam Review Dataset using Apache Spark
 - Investigated 42 gigabyte dataset of Steam game reviews.
 - Performed Descriptive analysis of the dataset including average review percentage, genre-specific review averages and review trends over time.
 - Performed Technical analysis using a Frequent Itemset algorithm to find trends at the individual user level with regards to frequently grouped genres and games.

Experience

- Acme: Jan 2023-Present
 - Collaborated with coworkers to efficiently complete a variety of tasks.
 - Adapted to a dynamic environment that required different roles for each shift.
 - Communicated clearly and concisely with customers.

Technical Skills

- C, Java, Python
- Microsoft Excel, Word, PowerPoint
- Apache Spark
- SPSS Statistical Software

Activities

- West Chester University Computer Science Club
 - Discussed concepts with more experienced students and learned about additional ways to engage with the field.
- Placed third in the West Chester University Programming Competition
- Executive Board of West Chester Fencing Club